



## **SIMATS ENGINEERING**



**Saveetha Institute of Medical and Technical Sciences**

**Chennai 2026**

# **ASSESSMENT TOOL -3**

## **CO4\_AT3**

**Name :** N.Sunil Kumar

**Register No :** 192472330

**Course code :** CSA6502

**Course name:** Generative Ai And Large Scale Models

**Faculty name:** Dr.Nithisha

## QUESTION :-

**Modify a text-to-image program to allow the user to repeatedly generate images until they select Exit.**

## ANSWER :-

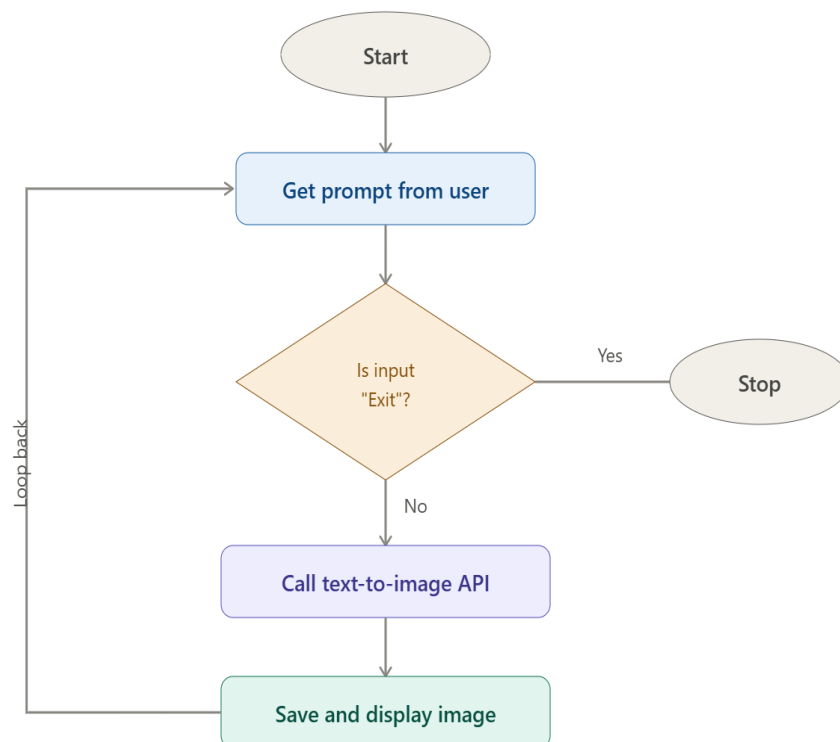
### Aim

To write a Python program that takes a text prompt from the user and generates an image using a text-to-image model, repeating this process in a loop until the user chooses to exit.

### Software Requirements

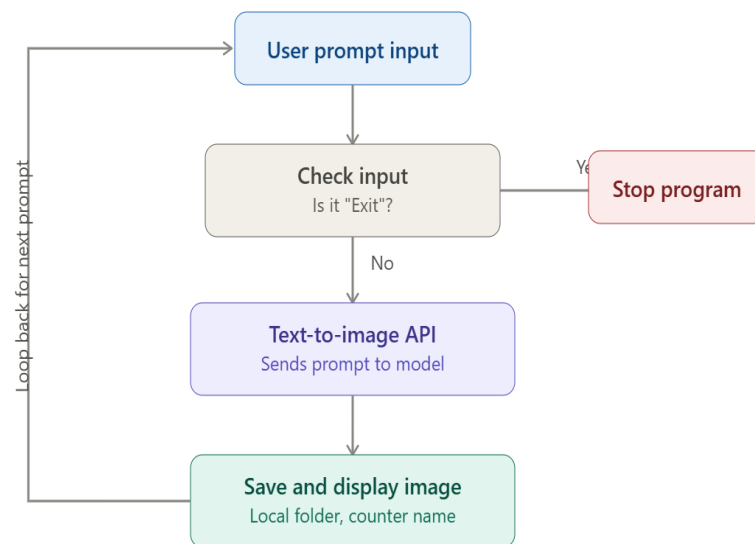
- Python 3.8 or above
- openai library (pip install openai)
- requests and Pillow libraries
- A valid API key for a text-to-image model (e.g. DALL·E, Stability AI)

### Architecture



The architecture consists of four main components: the input module that captures the user's prompt, a decision check that tests for the exit keyword, the API module that sends the prompt to the text-to-image model, and the storage module that saves and displays the generated image before looping back for the next prompt.

### Flowchart



### Algorithm

1. Start the program and initialize an image counter and output folder.
2. Prompt the user for a text description.
3. If input equals "Exit" (case-insensitive), terminate the loop.
4. Otherwise, send the prompt to the text-to-image API.
5. Handle errors gracefully (network failure, invalid prompt, rate limit).
6. Save the generated image locally with a unique filename.
7. Log the prompt and filename to a session record.
8. Display the image to the user (optional).
9. Repeat from step 2.
10. On exit, print a summary of all images generated in the session.

## Program

```
import openai

import os

import requests

from datetime import datetime

from PIL import Image

openai.api_key = os.getenv("OPENAI_API_KEY", "your-api-key-here")

OUTPUT_DIR = "generated_images"

os.makedirs(OUTPUT_DIR, exist_ok=True)

session_log = [] # keeps track of (prompt, filename) for this session

def generate_image(prompt, index):

    """Generate one image from a prompt, with basic error handling and retry."""

    max_retries = 2

    for attempt in range(1, max_retries + 1):

        try:

            response = openai.images.generate(

                model="dall-e-3",

                prompt=prompt,

                n=1,

                size="1024x1024" )

            image_url = response.data[0].url

            image_data = requests.get(image_url, timeout=15).content

            filename = os.path.join(OUTPUT_DIR, f'image_{index}.png')

            with open(filename, "wb") as f:

                f.write(image_data)
```

```

        session_log.append((prompt, filename))

        print(f'Saved: {filename}')

        try: Image.open(filename).show()

        except Exception:

            pass # display is optional, skip silently if it fails

        return filename

    except Exception as e:

        print(f'Attempt {attempt} failed: {e}')

        if attempt == max_retries:

            print("Skipping this prompt after repeated failures.")

            return None

def print_summary():

    print("\n=== Session Summary ===")

    if not session_log:

        print("No images were generated.")

        return

    for i, (prompt, filename) in enumerate(session_log, start=1):

        print(f'{i}. "{prompt}" -> {filename}')

    print(f'Total images generated: {len(session_log)}')

def main():

    print("=== Text-to-Image Generator ===")

    print("Type a prompt to generate an image, or type 'Exit' to quit.\n")

    index = 1

```

```
while True:

    prompt = input("Enter prompt: ").strip()

    if prompt.lower() == "exit":

        print("\nExiting program...")

        break

    if not prompt:

        print("Prompt cannot be empty.\n")

        continue

    generate_image(prompt, index)

    index += 1

    print()

    print_summary()

if __name__ == "__main__":

    main()
```

### **Sample Output**

=== Text-to-Image Generator ===

Type a prompt to generate an image, or type 'Exit' to quit.

Enter prompt: a sunset over mountains

Saved: generated\_images/image\_1.png

Enter prompt: a futuristic city at night

Saved: generated\_images/image\_2.png

Enter prompt: Exit

Exiting program...

### === Session Summary ===

1. "a sunset over mountains" -> generated\_images/image\_1.png

2. "a futuristic city at night" -> generated\_images/image\_2.png

Total images generated: 2

[Insert sample generated image 1 here — output for "a sunset over mountains"]

[Insert sample generated image 2 here — output for "a futuristic city at night"]

### Sample output image



### Advantages

- Loop-based design lets the user generate multiple images without restarting the program.
- Retry logic and error handling make the program robust to transient API failures.
- The session log gives a clear record of every prompt and the image it produced.

### Applications

- Rapid prototyping of concept art, marketing visuals, or storyboards.
- Educational demonstrations of generative AI capability.
- Batch-style creative brainstorming tools.

### Conclusion

Thus, a Python program was successfully designed and implemented to repeatedly accept text prompts from the user and generate corresponding images using a text-to-image model. The program loops continuously, allowing multiple images to be created in a single session, and terminates cleanly when the user types "Exit", displaying a summary of all images generated during the session

## **TEXT-TO-SPEECH PROGRAMMING**

**29. Write a program that checks whether the input text is empty before performing TTS.**

### **Aim**

To write a Python program that accepts text input from the user and checks whether it is empty before converting it into speech using Text-to-Speech (TTS).

### **Algorithm**

1. Start the program.
2. Import the required TTS library (pyttsx3).
3. Get the input text from the user.
4. Check whether the input text is empty (or contains only whitespace).
5. If the text is empty, display an error message and stop the process.
6. If the text is not empty, initialize the TTS engine and convert the text to speech.
7. Stop the program.

### **Program**

#### **python**

```
import pyttsx3

def text_to_speech(text):

    # Step 1: Check if the input text is empty

    if text is None or text.strip() == "":

        print("Error: Input text is empty. TTS cannot be performed.")

        return False

    # Step 2: Perform Text-to-Speech since input is valid

    engine = pyttsx3.init()

    engine.say(text)

    engine.runAndWait()
```



```
    print(f'Speaking: \"{text}\"')

    return True

def main():

    user_input = input("Enter text to convert to speech: ")

    result = text_to_speech(user_input)

    if not result:

        print("TTS process terminated due to empty input.")

if __name__ == "__main__":

    main()
```

## **Output**

Case 1: Empty input

Enter text to convert to speech:

Error: Input text is empty. TTS cannot be performed.

TTS process terminated due to empty input.

Case 2: Valid input

Enter text to convert to speech: Hello, welcome to Python programming

Speaking: "Hello, welcome to Python programming"

## **Result**

Thus, a Python program to check whether the input text is empty before performing Text-to-Speech was written and executed successfully.